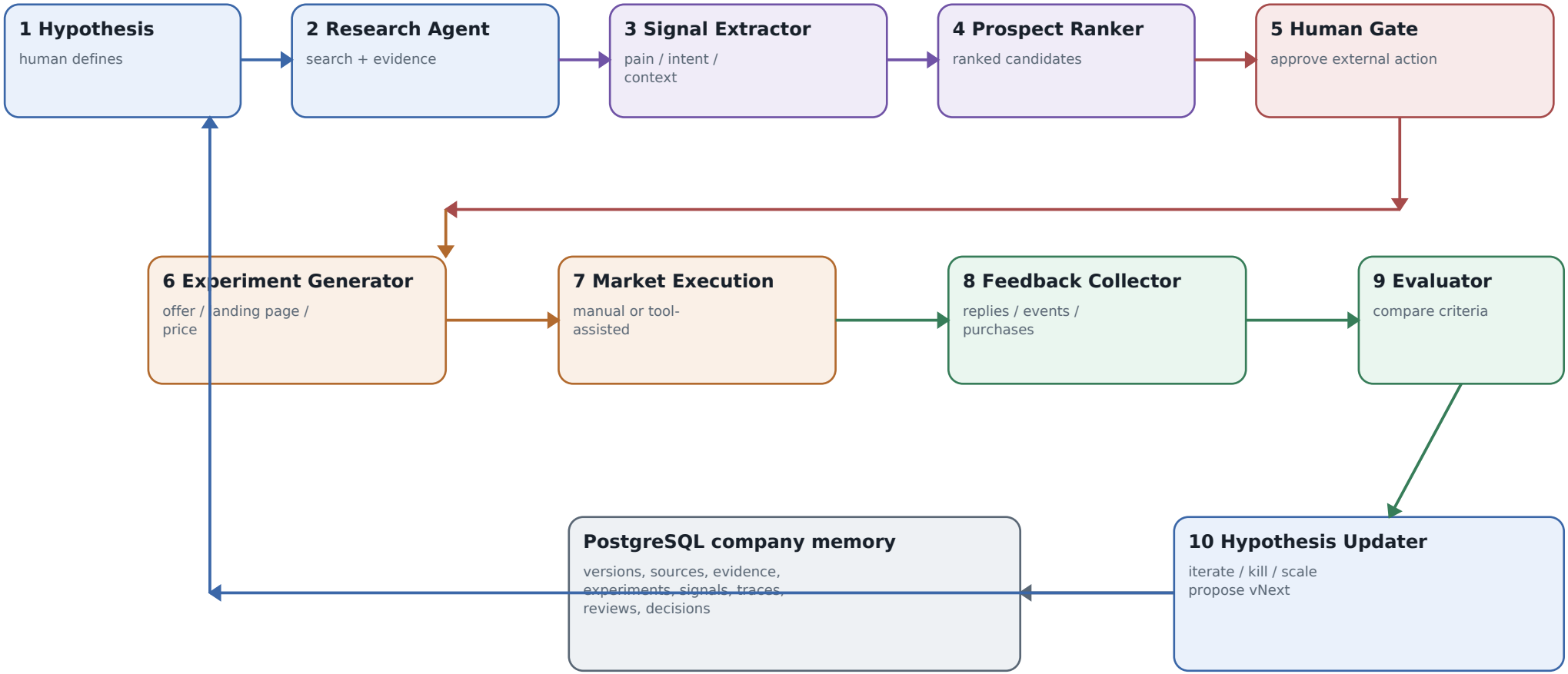
Hypothesis Loop - System & Agent Interaction

Architektur eines Closed-Loop-Systems fuer Ideenfindung, Markttests und iteratives Lernen.

Architekturregel

Code steuert den Prozess. LLMs loesen unstrukturierte Teilaufgaben. Menschen genehmigen riskante oder extern sichtbare Aktionen. PostgreSQL speichert den dauerhaften Zustand.

1. End-to-End-Loop


1 Hypothesis
human defines
2 Research Agent
search + evidence
3 Signal Extractor
pain / intent / context
4 Prospect Ranker
ranked candidates
5 Human Gate
approve external action
6 Experiment Generator
offer / landing page / price
7 Market Execution
manual or tool-assisted
8 Feedback Collector
replies / events / purchases
9 Evaluator
compare criteria
PostgreSQL company memory
versions, sources, evidence, experiments, signals, traces, reviews, decisions
10 Hypothesis Updater
iterate / kill / scale
propose vNext

2. Komponenten und I/O-Vertraege

Orchestrator	Workflow-State, Approvals, Run-Status	State-Transitions, Task-Dispatch	Marktfakten erfinden oder Gates ueberspringen
Research Agent	hypothesis_version, Tool-Policy	sources, evidence candidates	Personen kontaktieren oder Hypothese veraendern
Signal Extractor	Source/Evidence Text	Pain, Intent, Kontext, Stance, Strength	LLM confidence als Kaufsignal behandeln
Prospect Ranker	Evidence + public context	ranked prospects + Evidence-IDs	behaupten, jemand werde kaufen
Experiment Generator	Hypothese, Evidenz, Muster	Testvorschlag, Offer, Assets, Drafts	ohne Freigabe publizieren
Feedback Collector	Events, Replies, Notes	signals + qualitative feedback	Rohdaten in Erfolg umdeuten
Evaluator / Updater	Kriterien, Evidenz, Signals	iterate/kill/scale + vNext proposal	alte Versionen ueberschreiben
Human	Agent-Proposals + Evidence	Approval, Modification, Decision	aus Audit Trail verschwinden

3. Empfohlene Service-Grenzen

api	FastAPI Endpoints, DTO Validation, Human Approval Endpoints
domain	Entities, Enums, State Rules, Business Invariants
repositories	PostgreSQL persistence + transactions
orchestrator	Next-step calculation, dispatch, retry, idempotency
agents/research	Search planning, source triage, evidence extraction
agents/experiment	Prospect ranking, test and offer generation
agents/evaluation	Signal analysis, update proposal, rationale
tools	Search/browser/analytics/email adapters
tracing	agent_run lifecycle, model/tool metadata, cost/errors

4. MVP-Reihenfolge

M0	Repo + FastAPI + Postgres + Migrations + Tests
M1	Hypothesis v1 -> Research Agent -> Sources/Evidence persistieren
M2	Evaluator liest Evidence -> v2 / kill / continue Vorschlag mit Record-IDs
M3	Human approval gate + Audit Trail
M4	Experiment + Signal Tabellen fuer einen manuellen Markttest
M5	Prospecting + Experiment Generation
M6	UI fuer State, Evidence, Runs und Decisions

5. Kompakter Kontext fuer Codex

System goal

Build a closed-loop hypothesis testing platform for AI-native business discovery.

Non-negotiables:
1. PostgreSQL is the durable company memory.
2. Hypotheses are versioned and never overwritten after testing begins.
3. Real market observations are stored as signals; model opinions are not signals.
4. Orchestration is deterministic in V1.
5. LLM components receive narrow, typed inputs and return schema-validated outputs.
6. Public/outbound actions require human approval in V1.
7. Evaluator recommendations cite persisted evidence/signal IDs.
8. Every agent execution is traceable through agent_run.

First vertical slice

Create hypothesis v1 -> run research -> store sources/evidence -> evaluate evidence -> produce a reviewable next-action proposal.